Group14_Source_Code7

May 24, 2026

1 Hybrid Convolutional Feature Extraction Ensembled with Extreme Gradient Boosting (XGBoost)

[1]:
```
# This Python 3 environment comes with many helpful analytics libraries␣
 ↪installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/
 ↪docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list␣
 ↪all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that␣
 ↪gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved␣
 ↪outside of the current session

# Use the kagglehub client library to attach Kaggle resources like␣
 ↪competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/
 ↪README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/sachin211104/sachin-chess-project/chess_tensor.npy
/kaggle/input/datasets/sachin211104/sachin-chess-project/chess_scalar.parquet
```

[3]:
```
# ============================================================
# CHESS HYBRID MODEL
# CNN FEATURE EXTRACTOR + XGBOOST CLASSIFIER
# KAGGLE VERSION (RAM SAFE + CHECKPOINT SAFE)
# ============================================================

# ============================================================
# INSTALL XGBOOST
# ============================================================
!pip install xgboost -q

# ============================================================
# IMPORTS
# ============================================================
import os
import gc
import joblib

import numpy as np
import pandas as pd

import tensorflow as tf
from tensorflow.keras import layers, models, regularizers

from sklearn.utils import resample
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV

import matplotlib.pyplot as plt
import seaborn as sns

# ============================================================
# PATHS
# ============================================================

# CHANGE THIS TO YOUR KAGGLE DATASET NAME
base_path = "/kaggle/input/datasets/sachin211104/sachin-chess-project/"

# writable directory
```

```
save_path = "/kaggle/working/"

# ============================================================
# LOAD DATA
# ============================================================

print("\nLoading scalar dataframe...\n")

df = pd.read_parquet(
    base_path + "chess_scalar.parquet"
)

print(df.shape)

print("\nLoading tensor data using memory mapping...\n")

X_tensor = np.load(
    base_path + "chess_tensor.npy",
    mmap_mode='r'
)

print(X_tensor.shape)

# ============================================================
# ORIGINAL CLASS COUNTS
# ============================================================

print("\n========== ORIGINAL CLASS COUNTS ==========\n")
print(df["label"].value_counts())

# ============================================================
# CREATE INDEX COLUMN
# ============================================================

df = df.reset_index(drop=True)

df["idx"] = np.arange(len(df))

# ============================================================
# OVERSAMPLING
# ============================================================

print("\nPerforming oversampling...\n")

max_count = df["label"].value_counts().max()

balanced_indices = []
```

```
for label_name in df["label"].unique():

    class_indices = df[
        df["label"] == label_name
    ]["idx"].values

    oversampled_indices = resample(
        class_indices,
        replace=True,
        n_samples=max_count,
        random_state=42
    )

    balanced_indices.extend(
        oversampled_indices
    )

balanced_indices = np.array(
    balanced_indices
)

np.random.shuffle(
    balanced_indices
)

# ============================================================
# BUILD BALANCED DATA
# ============================================================

df_balanced = df.iloc[
    balanced_indices
].reset_index(drop=True)

X_tensor_balanced = X_tensor[
    balanced_indices
]

print("\n========== BALANCED CLASS COUNTS ==========\n")
print(df_balanced["label"].value_counts())

# ============================================================
# LABEL ENCODING
# ============================================================

le = LabelEncoder()
```

```
y_encoded = le.fit_transform(
    df_balanced["label"]
)

y = tf.keras.utils.to_categorical(
    y_encoded
)

print("\nClasses:\n")
print(le.classes_)

# ============================================================
# REMOVE LEAKAGE FEATURES
# ============================================================

leak_cols = [
    "label",
    "delta",
    "eval",
    "piece_moved",
    "is_capture",
    "gives_check",
    "idx"
]

X_scalar = df_balanced.drop(
    columns=leak_cols
).values

print("\nScalar shape:")
print(X_scalar.shape)

# ============================================================
# TRAIN TEST SPLIT
# ============================================================

print("\nSplitting dataset...\n")

X_tensor_train, X_tensor_test, \
X_scalar_train, X_scalar_test, \
y_train, y_test = train_test_split(
    X_tensor_balanced,
    X_scalar,
    y,
    test_size=0.2,
    random_state=42,
    stratify=np.argmax(y, axis=1)
```

```
)

# ============================================================
# SCALE SCALAR FEATURES
# ============================================================

print("\nScaling scalar features...\n")

scaler = StandardScaler()

X_scalar_train = scaler.fit_transform(
    X_scalar_train
)

X_scalar_test = scaler.transform(
    X_scalar_test
)

# ============================================================
# MEMORY OPTIMIZATION
# ============================================================

print("\nApplying memory optimization...\n")

X_tensor_train = X_tensor_train.astype(
    np.float16
)

X_tensor_test = X_tensor_test.astype(
    np.float16
)

X_scalar_train = X_scalar_train.astype(
    np.float32
)

X_scalar_test = X_scalar_test.astype(
    np.float32
)

# ============================================================
# CNN FEATURE EXTRACTOR
# ============================================================

def build_cnn_model(
    tensor_shape,
    scalar_dim,
```

```
    num_classes
):

    # ----------------------------
    # TENSOR INPUT
    # ----------------------------

    tensor_input = layers.Input(
        shape=tensor_shape
    )

    x = layers.Conv2D(
        32,
        (3,3),
        padding='same',
        activation='swish'
    )(tensor_input)

    x = layers.BatchNormalization()(x)

    x = layers.MaxPooling2D((2,2))(x)

    x = layers.Conv2D(
        64,
        (3,3),
        padding='same',
        activation='swish'
    )(x)

    x = layers.BatchNormalization()(x)

    x = layers.Flatten()(x)

    cnn_features = layers.Dense(
        128,
        activation='swish',
        kernel_regularizer=regularizers.l2(0.001),
        name="deep_features"
    )(x)

    x = layers.Dropout(0.3)(
        cnn_features
    )

    # ----------------------------
    # SCALAR INPUT
    # ----------------------------
```

```
scalar_input = layers.Input(
    shape=(scalar_dim,)
)

y = layers.Dense(
    64,
    activation='swish'
)(scalar_input)

y = layers.BatchNormalization()(y)

y = layers.Dropout(0.2)(y)

# ----------------------------
# MERGE
# ----------------------------

combined = layers.concatenate(
    [x, y]
)

z = layers.Dense(
    128,
    activation='swish'
)(combined)

z = layers.Dropout(0.3)(z)

output = layers.Dense(
    num_classes,
    activation='softmax'
)(z)

model = models.Model(
    inputs=[tensor_input, scalar_input],
    outputs=output
)

optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0005,
    clipnorm=1.0
)

model.compile(
    optimizer=optimizer,
    loss='categorical_crossentropy',
```

```
        metrics=['accuracy']
    )

    return model

# ============================================================
# BUILD MODEL
# ============================================================

cnn_model = build_cnn_model(
    tensor_shape=X_tensor_train.shape[1:],
    scalar_dim=X_scalar_train.shape[1],
    num_classes=len(le.classes_)
)

cnn_model.summary()

# ============================================================
# CALLBACKS
# ============================================================

checkpoint_path = (
    save_path + "best_cnn_model.keras"
)

checkpoint = tf.keras.callbacks.ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='val_loss',
    save_best_only=True,
    verbose=1
)

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

# ============================================================
# TRAIN CNN
# ============================================================

print("\nTraining CNN...\n")

history = cnn_model.fit(
    [X_tensor_train, X_scalar_train],
    y_train,
```

```
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[checkpoint, early_stop],
    verbose=1
)

# ============================================================
# SAVE FINAL CNN
# ============================================================

cnn_model.save(
    save_path + "final_cnn_model.keras"
)

print("\nCNN model saved.\n")

# ============================================================
# FEATURE EXTRACTOR MODEL
# ============================================================

feature_extractor = models.Model(
    inputs=cnn_model.inputs,
    outputs=cnn_model.get_layer(
        "deep_features"
    ).output
)

# ============================================================
# FEATURE EXTRACTION FUNCTION
# ============================================================

def extract_features_in_batches(
    tensor_data,
    scalar_data,
    batch_size=1024
):

    features = []

    total = len(tensor_data)

    for start in range(0, total, batch_size):

        end = min(start + batch_size, total)

        batch_tensor = tensor_data[start:end]
```

```
        batch_scalar = scalar_data[start:end]

        batch_features = feature_extractor.predict(
            [batch_tensor, batch_scalar],
            verbose=0
        )

        features.append(batch_features)

        if start % (batch_size * 20) == 0:
            print(
                f"Processed {end}/{total}"
            )

        gc.collect()

    return np.vstack(features)

# ============================================================
# EXTRACT TRAIN FEATURES
# ============================================================

print("\nExtracting TRAIN deep features...\n")

train_cnn_features = extract_features_in_batches(
    X_tensor_train,
    X_scalar_train,
    batch_size=1024
)

# ============================================================
# EXTRACT TEST FEATURES
# ============================================================

print("\nExtracting TEST deep features...\n")

test_cnn_features = extract_features_in_batches(
    X_tensor_test,
    X_scalar_test,
    batch_size=1024
)

# ============================================================
# SAVE FEATURES
# ============================================================
```

```
np.save(
    save_path + "train_cnn_features.npy",
    train_cnn_features
)

np.save(
    save_path + "test_cnn_features.npy",
    test_cnn_features
)

print("\nCNN features saved.\n")

# ============================================================
# COMBINE FEATURES FOR XGBOOST
# ============================================================

X_train_hybrid = np.concatenate(
    [train_cnn_features, X_scalar_train],
    axis=1
)

X_test_hybrid = np.concatenate(
    [test_cnn_features, X_scalar_test],
    axis=1
)

# ============================================================
# FREE MEMORY
# ============================================================

del train_cnn_features
del test_cnn_features

gc.collect()

# ============================================================
# LABELS FOR XGBOOST
# ============================================================

y_train_labels = np.argmax(
    y_train,
    axis=1
)

y_test_labels = np.argmax(
    y_test,
    axis=1
```

```
)

# ============================================================
# BASE XGBOOST MODEL
# ============================================================

xgb = XGBClassifier(
    objective='multi:softprob',
    num_class=len(le.classes_),
    tree_method='hist',
    device='cuda',
    eval_metric='mlogloss',
    random_state=42
)

# ============================================================
# HYPERPARAMETER SPACE
# ============================================================

param_grid = {

    'n_estimators': [300, 500],

    'max_depth': [6, 8, 10],

    'learning_rate': [0.03, 0.05],

    'subsample': [0.8, 1.0],

    'colsample_bytree': [0.8, 1.0],

    'gamma': [0, 0.1],

    'reg_alpha': [0, 0.5],

    'reg_lambda': [1, 2]
}

# ============================================================
# RANDOM SEARCH
# ============================================================

print("\nStarting XGBoost hyperparameter tuning...\n")

random_search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_grid,
```

```
    n_iter=10,
    scoring='accuracy',
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=1
)

random_search.fit(
    X_train_hybrid,
    y_train_labels
)

# ============================================================
# BEST MODEL
# ============================================================

best_xgb = random_search.best_estimator_

print("\nBest Parameters:\n")
print(random_search.best_params_)

# ============================================================
# SAVE XGBOOST MODEL
# ============================================================

best_xgb.save_model(
    save_path + "best_hybrid_xgb.json"
)

joblib.dump(
    best_xgb,
    save_path + "best_hybrid_xgb.pkl"
)

print("\nXGBoost model saved.\n")

# ============================================================
# PREDICTIONS
# ============================================================

print("\nEvaluating model...\n")

y_pred = best_xgb.predict(
    X_test_hybrid
)
```

```
# ============================================================
# ACCURACY
# ============================================================

accuracy = accuracy_score(
    y_test_labels,
    y_pred
)

print(
    f"\nHybrid Model Accuracy: {accuracy*100:.2f}%"
)

# ============================================================
# CONFUSION MATRIX
# ============================================================

plt.figure(figsize=(8,6))

cm = confusion_matrix(
    y_test_labels,
    y_pred
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=le.classes_,
    yticklabels=le.classes_
)

plt.title(
    'Hybrid CNN + XGBoost Confusion Matrix'
)

plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.tight_layout()

plt.show()

# ============================================================
# CLASSIFICATION REPORT
# ============================================================
```

```
print("\nClassification Report:\n")

print(
    classification_report(
        y_test_labels,
        y_pred,
        target_names=le.classes_
    )
)

# ============================================================
# SAVE SCALER + LABEL ENCODER
# ============================================================

joblib.dump(
    scaler,
    save_path + "scaler.pkl"
)

joblib.dump(
    le,
    save_path + "label_encoder.pkl"
)

print("\nEverything saved successfully.\n")

print("\nSaved files location:\n")
print(save_path)
```

```
Loading scalar dataframe…

(337981, 21)

Loading tensor data using memory mapping…

(337981, 12, 8, 8)

========== ORIGINAL CLASS COUNTS ==========

label
good          221151
excellent      57825
inaccuracy     28194
mistake        22817
blunder         7994
Name: count, dtype: int64
```

```
Performing oversampling…


========== BALANCED CLASS COUNTS ==========

label
mistake       221151
inaccuracy    221151
excellent     221151
blunder       221151
good          221151
Name: count, dtype: int64

Classes:

['blunder' 'excellent' 'good' 'inaccuracy' 'mistake']

Scalar shape:
(1105755, 15)

Splitting dataset…


Scaling scalar features…


Applying memory optimization…


I0000 00:00:1779475317.452581      57 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 13757 MB memory:  -> device:
0, name: Tesla T4, pci bus id: 0000:00:04.0, compute capability: 7.5
I0000 00:00:1779475317.458572      57 gpu_device.cc:2019] Created device
/job:localhost/replica:0/task:0/device:GPU:1 with 13757 MB memory:  -> device:
1, name: Tesla T4, pci bus id: 0000:00:05.0, compute capability: 7.5
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 12, 8, 8)	0	-
conv2d (Conv2D)	(None, 12, 8, 32)	2,336	input_layer[0][0]
batch_normalization	(None, 12, 8, 32)	128	conv2d[0][0]

```
 (BatchNormalizatio…

 max_pooling2d          (None, 6, 4, 32)            0   batch_normalizat…
 (MaxPooling2D)

 conv2d_1 (Conv2D)      (None, 6, 4, 64)       18,496   max_pooling2d[0]…

 batch_normalizatio…    (None, 6, 4, 64)          256   conv2d_1[0][0]
 (BatchNormalizatio…

 input_layer_1          (None, 15)                  0   -
 (InputLayer)

 flatten (Flatten)      (None, 1536)                0   batch_normalizat…

 dense (Dense)          (None, 64)              1,024   input_layer_1[0]…

 deep_features          (None, 128)           196,736   flatten[0][0]
 (Dense)

 batch_normalizatio…    (None, 64)                256   dense[0][0]
 (BatchNormalizatio…

 dropout (Dropout)      (None, 128)                 0   deep_features[0]…

 dropout_1 (Dropout)    (None, 64)                  0   batch_normalizat…

 concatenate            (None, 192)                 0   dropout[0][0],
 (Concatenate)                                          dropout_1[0][0]

 dense_1 (Dense)        (None, 128)            24,704   concatenate[0][0]

 dropout_2 (Dropout)    (None, 128)                 0   dense_1[0][0]

 dense_2 (Dense)        (None, 5)                 645   dropout_2[0][0]


 Total params: 244,581 (955.39 KB)

 Trainable params: 244,261 (954.14 KB)

 Non-trainable params: 320 (1.25 KB)


Training CNN…
```

```
Epoch 1/100

WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1779475331.150458     147 service.cc:152] XLA service 0x7e7e34044c00
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1779475331.150561     147 service.cc:160]   StreamExecutor device
(0): Tesla T4, Compute Capability 7.5
I0000 00:00:1779475331.150566     147 service.cc:160]   StreamExecutor device
(1): Tesla T4, Compute Capability 7.5
I0000 00:00:1779475332.226859     147 cuda_dnn.cc:529] Loaded cuDNN version
91002

   48/22116            1:12 3ms/step -
accuracy: 0.2580 - loss: 2.0330

I0000 00:00:1779475338.208192     147 device_compiler.h:188] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.

22116/22116            0s 3ms/step -
accuracy: 0.4729 - loss: 1.2199
Epoch 1: val_loss improved from inf to 1.07560, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            101s 4ms/step
- accuracy: 0.4729 - loss: 1.2199 - val_accuracy: 0.5499 - val_loss: 1.0756
Epoch 2/100
22114/22116            0s 3ms/step -
accuracy: 0.5763 - loss: 1.0560
Epoch 2: val_loss improved from 1.07560 to 0.99638, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.5763 - loss: 1.0560 - val_accuracy: 0.6123 - val_loss: 0.9964
Epoch 3/100
22106/22116            0s 3ms/step -
accuracy: 0.6129 - loss: 1.0075
Epoch 3: val_loss improved from 0.99638 to 0.96199, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.6129 - loss: 1.0075 - val_accuracy: 0.6401 - val_loss: 0.9620
Epoch 4/100
22111/22116            0s 3ms/step -
accuracy: 0.6362 - loss: 0.9791
Epoch 4: val_loss improved from 0.96199 to 0.94853, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6362 - loss: 0.9791 - val_accuracy: 0.6560 - val_loss: 0.9485
Epoch 5/100
22108/22116            0s 3ms/step -
accuracy: 0.6546 - loss: 0.9626
```

```
Epoch 5: val_loss improved from 0.94853 to 0.92967, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6546 - loss: 0.9626 - val_accuracy: 0.6698 - val_loss: 0.9297
Epoch 6/100
22103/22116            0s 3ms/step -
accuracy: 0.6642 - loss: 0.9476
Epoch 6: val_loss improved from 0.92967 to 0.92104, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6642 - loss: 0.9476 - val_accuracy: 0.6763 - val_loss: 0.9210
Epoch 7/100
22103/22116            0s 3ms/step -
accuracy: 0.6732 - loss: 0.9379
Epoch 7: val_loss improved from 0.92104 to 0.90665, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.6732 - loss: 0.9379 - val_accuracy: 0.6860 - val_loss: 0.9066
Epoch 8/100
22101/22116            0s 3ms/step -
accuracy: 0.6787 - loss: 0.9279
Epoch 8: val_loss improved from 0.90665 to 0.89651, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6787 - loss: 0.9279 - val_accuracy: 0.6905 - val_loss: 0.8965
Epoch 9/100
22115/22116            0s 3ms/step -
accuracy: 0.6828 - loss: 0.9213
Epoch 9: val_loss improved from 0.89651 to 0.89520, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6828 - loss: 0.9213 - val_accuracy: 0.6944 - val_loss: 0.8952
Epoch 10/100
22107/22116            0s 3ms/step -
accuracy: 0.6877 - loss: 0.9128
Epoch 10: val_loss improved from 0.89520 to 0.88052, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.6877 - loss: 0.9128 - val_accuracy: 0.7019 - val_loss: 0.8805
Epoch 11/100
22104/22116            0s 3ms/step -
accuracy: 0.6911 - loss: 0.9088
Epoch 11: val_loss did not improve from 0.88052
22116/22116            84s 4ms/step
- accuracy: 0.6911 - loss: 0.9088 - val_accuracy: 0.7009 - val_loss: 0.8825
Epoch 12/100
22104/22116            0s 3ms/step -
accuracy: 0.6915 - loss: 0.9059
```

```
Epoch 12: val_loss improved from 0.88052 to 0.87663, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.6915 - loss: 0.9059 - val_accuracy: 0.7052 - val_loss: 0.8766
Epoch 13/100
22102/22116            0s 3ms/step -
accuracy: 0.6958 - loss: 0.8984
Epoch 13: val_loss improved from 0.87663 to 0.87590, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6958 - loss: 0.8984 - val_accuracy: 0.7052 - val_loss: 0.8759
Epoch 14/100
22113/22116            0s 3ms/step -
accuracy: 0.6968 - loss: 0.8963
Epoch 14: val_loss improved from 0.87590 to 0.86804, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            85s 4ms/step
- accuracy: 0.6968 - loss: 0.8963 - val_accuracy: 0.7094 - val_loss: 0.8680
Epoch 15/100
22100/22116            0s 3ms/step -
accuracy: 0.6983 - loss: 0.8938
Epoch 15: val_loss improved from 0.86804 to 0.86050, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.6983 - loss: 0.8938 - val_accuracy: 0.7150 - val_loss: 0.8605
Epoch 16/100
22110/22116            0s 3ms/step -
accuracy: 0.7004 - loss: 0.8892
Epoch 16: val_loss did not improve from 0.86050
22116/22116            84s 4ms/step
- accuracy: 0.7004 - loss: 0.8892 - val_accuracy: 0.7116 - val_loss: 0.8648
Epoch 17/100
22116/22116            0s 3ms/step -
accuracy: 0.7028 - loss: 0.8857
Epoch 17: val_loss did not improve from 0.86050
22116/22116            84s 4ms/step
- accuracy: 0.7028 - loss: 0.8857 - val_accuracy: 0.7127 - val_loss: 0.8618
Epoch 18/100
22107/22116            0s 3ms/step -
accuracy: 0.7031 - loss: 0.8835
Epoch 18: val_loss improved from 0.86050 to 0.84996, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.7031 - loss: 0.8835 - val_accuracy: 0.7165 - val_loss: 0.8500
Epoch 19/100
22106/22116            0s 3ms/step -
accuracy: 0.7057 - loss: 0.8803
Epoch 19: val_loss did not improve from 0.84996
```

```
22116/22116            83s 4ms/step
- accuracy: 0.7057 - loss: 0.8803 - val_accuracy: 0.7158 - val_loss: 0.8523
Epoch 20/100
22107/22116            0s 3ms/step -
accuracy: 0.7060 - loss: 0.8784
Epoch 20: val_loss improved from 0.84996 to 0.84700, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.7060 - loss: 0.8784 - val_accuracy: 0.7205 - val_loss: 0.8470
Epoch 21/100
22111/22116            0s 3ms/step -
accuracy: 0.7074 - loss: 0.8759
Epoch 21: val_loss did not improve from 0.84700
22116/22116            83s 4ms/step
- accuracy: 0.7074 - loss: 0.8759 - val_accuracy: 0.7200 - val_loss: 0.8479
Epoch 22/100
22111/22116            0s 3ms/step -
accuracy: 0.7080 - loss: 0.8753
Epoch 22: val_loss improved from 0.84700 to 0.84588, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.7080 - loss: 0.8753 - val_accuracy: 0.7204 - val_loss: 0.8459
Epoch 23/100
22112/22116            0s 3ms/step -
accuracy: 0.7091 - loss: 0.8720
Epoch 23: val_loss improved from 0.84588 to 0.84297, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.7091 - loss: 0.8720 - val_accuracy: 0.7205 - val_loss: 0.8430
Epoch 24/100
22104/22116            0s 3ms/step -
accuracy: 0.7107 - loss: 0.8701
Epoch 24: val_loss improved from 0.84297 to 0.83942, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.7107 - loss: 0.8701 - val_accuracy: 0.7239 - val_loss: 0.8394
Epoch 25/100
22106/22116            0s 3ms/step -
accuracy: 0.7113 - loss: 0.8702
Epoch 25: val_loss did not improve from 0.83942
22116/22116            84s 4ms/step
- accuracy: 0.7113 - loss: 0.8702 - val_accuracy: 0.7220 - val_loss: 0.8401
Epoch 26/100
22103/22116            0s 3ms/step -
accuracy: 0.7122 - loss: 0.8675
Epoch 26: val_loss did not improve from 0.83942
22116/22116            85s 4ms/step
- accuracy: 0.7122 - loss: 0.8675 - val_accuracy: 0.7208 - val_loss: 0.8444
```

```
Epoch 27/100
22103/22116             0s 3ms/step -
accuracy: 0.7142 - loss: 0.8625
Epoch 27: val_loss did not improve from 0.83942
22116/22116             87s 4ms/step
- accuracy: 0.7142 - loss: 0.8625 - val_accuracy: 0.7259 - val_loss: 0.8412
Epoch 28/100
22115/22116             0s 3ms/step -
accuracy: 0.7125 - loss: 0.8650
Epoch 28: val_loss improved from 0.83942 to 0.83362, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             83s 4ms/step
- accuracy: 0.7125 - loss: 0.8650 - val_accuracy: 0.7253 - val_loss: 0.8336
Epoch 29/100
22103/22116             0s 3ms/step -
accuracy: 0.7143 - loss: 0.8610
Epoch 29: val_loss did not improve from 0.83362
22116/22116             84s 4ms/step
- accuracy: 0.7143 - loss: 0.8610 - val_accuracy: 0.7259 - val_loss: 0.8377
Epoch 30/100
22107/22116             0s 3ms/step -
accuracy: 0.7145 - loss: 0.8620
Epoch 30: val_loss did not improve from 0.83362
22116/22116             84s 4ms/step
- accuracy: 0.7145 - loss: 0.8620 - val_accuracy: 0.7249 - val_loss: 0.8340
Epoch 31/100
22115/22116             0s 3ms/step -
accuracy: 0.7153 - loss: 0.8589
Epoch 31: val_loss improved from 0.83362 to 0.82587, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             85s 4ms/step
- accuracy: 0.7153 - loss: 0.8589 - val_accuracy: 0.7278 - val_loss: 0.8259
Epoch 32/100
22108/22116             0s 3ms/step -
accuracy: 0.7157 - loss: 0.8588
Epoch 32: val_loss did not improve from 0.82587
22116/22116             88s 4ms/step
- accuracy: 0.7157 - loss: 0.8588 - val_accuracy: 0.7268 - val_loss: 0.8339
Epoch 33/100
22101/22116             0s 3ms/step -
accuracy: 0.7164 - loss: 0.8566
Epoch 33: val_loss did not improve from 0.82587
22116/22116             85s 4ms/step
- accuracy: 0.7164 - loss: 0.8566 - val_accuracy: 0.7266 - val_loss: 0.8286
Epoch 34/100
22102/22116             0s 3ms/step -
accuracy: 0.7151 - loss: 0.8580
Epoch 34: val_loss did not improve from 0.82587
```

```
22116/22116            83s 4ms/step
- accuracy: 0.7151 - loss: 0.8580 - val_accuracy: 0.7272 - val_loss: 0.8282
Epoch 35/100
22113/22116            0s 3ms/step -
accuracy: 0.7179 - loss: 0.8554
Epoch 35: val_loss improved from 0.82587 to 0.82506, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.7179 - loss: 0.8554 - val_accuracy: 0.7287 - val_loss: 0.8251
Epoch 36/100
22109/22116            0s 3ms/step -
accuracy: 0.7177 - loss: 0.8548
Epoch 36: val_loss improved from 0.82506 to 0.82494, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            85s 4ms/step
- accuracy: 0.7177 - loss: 0.8548 - val_accuracy: 0.7283 - val_loss: 0.8249
Epoch 37/100
22110/22116            0s 3ms/step -
accuracy: 0.7174 - loss: 0.8534
Epoch 37: val_loss did not improve from 0.82494
22116/22116            87s 4ms/step
- accuracy: 0.7174 - loss: 0.8535 - val_accuracy: 0.7293 - val_loss: 0.8265
Epoch 38/100
22104/22116            0s 3ms/step -
accuracy: 0.7186 - loss: 0.8527
Epoch 38: val_loss improved from 0.82494 to 0.82413, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            86s 4ms/step
- accuracy: 0.7186 - loss: 0.8527 - val_accuracy: 0.7293 - val_loss: 0.8241
Epoch 39/100
22106/22116            0s 3ms/step -
accuracy: 0.7188 - loss: 0.8522
Epoch 39: val_loss improved from 0.82413 to 0.82323, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            84s 4ms/step
- accuracy: 0.7188 - loss: 0.8522 - val_accuracy: 0.7306 - val_loss: 0.8232
Epoch 40/100
22113/22116            0s 3ms/step -
accuracy: 0.7192 - loss: 0.8500
Epoch 40: val_loss did not improve from 0.82323
22116/22116            84s 4ms/step
- accuracy: 0.7192 - loss: 0.8500 - val_accuracy: 0.7290 - val_loss: 0.8244
Epoch 41/100
22103/22116            0s 3ms/step -
accuracy: 0.7204 - loss: 0.8482
Epoch 41: val_loss did not improve from 0.82323
22116/22116            83s 4ms/step
- accuracy: 0.7204 - loss: 0.8482 - val_accuracy: 0.7280 - val_loss: 0.8268
```

```
Epoch 42/100
22114/22116             0s 3ms/step -
accuracy: 0.7190 - loss: 0.8503
Epoch 42: val_loss improved from 0.82323 to 0.82264, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             85s 4ms/step
- accuracy: 0.7190 - loss: 0.8503 - val_accuracy: 0.7303 - val_loss: 0.8226
Epoch 43/100
22102/22116             0s 3ms/step -
accuracy: 0.7192 - loss: 0.8489
Epoch 43: val_loss improved from 0.82264 to 0.82077, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             84s 4ms/step
- accuracy: 0.7192 - loss: 0.8489 - val_accuracy: 0.7323 - val_loss: 0.8208
Epoch 44/100
22114/22116             0s 3ms/step -
accuracy: 0.7199 - loss: 0.8456
Epoch 44: val_loss did not improve from 0.82077
22116/22116             88s 4ms/step
- accuracy: 0.7199 - loss: 0.8456 - val_accuracy: 0.7302 - val_loss: 0.8252
Epoch 45/100
22102/22116             0s 3ms/step -
accuracy: 0.7208 - loss: 0.8457
Epoch 45: val_loss improved from 0.82077 to 0.81455, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             83s 4ms/step
- accuracy: 0.7208 - loss: 0.8457 - val_accuracy: 0.7342 - val_loss: 0.8146
Epoch 46/100
22107/22116             0s 3ms/step -
accuracy: 0.7212 - loss: 0.8445
Epoch 46: val_loss did not improve from 0.81455
22116/22116             83s 4ms/step
- accuracy: 0.7212 - loss: 0.8445 - val_accuracy: 0.7305 - val_loss: 0.8199
Epoch 47/100
22112/22116             0s 3ms/step -
accuracy: 0.7215 - loss: 0.8441
Epoch 47: val_loss did not improve from 0.81455
22116/22116             84s 4ms/step
- accuracy: 0.7215 - loss: 0.8441 - val_accuracy: 0.7341 - val_loss: 0.8174
Epoch 48/100
22108/22116             0s 3ms/step -
accuracy: 0.7217 - loss: 0.8436
Epoch 48: val_loss improved from 0.81455 to 0.81283, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             82s 4ms/step
- accuracy: 0.7217 - loss: 0.8436 - val_accuracy: 0.7338 - val_loss: 0.8128
Epoch 49/100
22103/22116             0s 3ms/step -
```

```
accuracy: 0.7220 - loss: 0.8430
Epoch 49: val_loss did not improve from 0.81283
22116/22116            82s 4ms/step
- accuracy: 0.7220 - loss: 0.8430 - val_accuracy: 0.7304 - val_loss: 0.8185
Epoch 50/100
22116/22116            0s 3ms/step -
accuracy: 0.7219 - loss: 0.8426
Epoch 50: val_loss did not improve from 0.81283
22116/22116            83s 4ms/step
- accuracy: 0.7219 - loss: 0.8426 - val_accuracy: 0.7326 - val_loss: 0.8161
Epoch 51/100
22115/22116            0s 3ms/step -
accuracy: 0.7226 - loss: 0.8407
Epoch 51: val_loss did not improve from 0.81283
22116/22116            83s 4ms/step
- accuracy: 0.7226 - loss: 0.8407 - val_accuracy: 0.7324 - val_loss: 0.8168
Epoch 52/100
22104/22116            0s 3ms/step -
accuracy: 0.7215 - loss: 0.8420
Epoch 52: val_loss improved from 0.81283 to 0.81002, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.7215 - loss: 0.8420 - val_accuracy: 0.7354 - val_loss: 0.8100
Epoch 53/100
22116/22116            0s 3ms/step -
accuracy: 0.7229 - loss: 0.8397
Epoch 53: val_loss did not improve from 0.81002
22116/22116            83s 4ms/step
- accuracy: 0.7229 - loss: 0.8397 - val_accuracy: 0.7320 - val_loss: 0.8145
Epoch 54/100
22108/22116            0s 3ms/step -
accuracy: 0.7232 - loss: 0.8391
Epoch 54: val_loss did not improve from 0.81002
22116/22116            83s 4ms/step
- accuracy: 0.7232 - loss: 0.8391 - val_accuracy: 0.7336 - val_loss: 0.8134
Epoch 55/100
22116/22116            0s 3ms/step -
accuracy: 0.7235 - loss: 0.8388
Epoch 55: val_loss improved from 0.81002 to 0.80964, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116            83s 4ms/step
- accuracy: 0.7235 - loss: 0.8388 - val_accuracy: 0.7352 - val_loss: 0.8096
Epoch 56/100
22115/22116            0s 3ms/step -
accuracy: 0.7232 - loss: 0.8385
Epoch 56: val_loss did not improve from 0.80964
22116/22116            83s 4ms/step
- accuracy: 0.7232 - loss: 0.8385 - val_accuracy: 0.7326 - val_loss: 0.8140
```

```
Epoch 57/100
22102/22116             0s 3ms/step -
accuracy: 0.7238 - loss: 0.8373
Epoch 57: val_loss did not improve from 0.80964
22116/22116             83s 4ms/step
- accuracy: 0.7238 - loss: 0.8373 - val_accuracy: 0.7352 - val_loss: 0.8102
Epoch 58/100
22113/22116             0s 3ms/step -
accuracy: 0.7246 - loss: 0.8367
Epoch 58: val_loss improved from 0.80964 to 0.80673, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             83s 4ms/step
- accuracy: 0.7246 - loss: 0.8367 - val_accuracy: 0.7372 - val_loss: 0.8067
Epoch 59/100
22102/22116             0s 3ms/step -
accuracy: 0.7245 - loss: 0.8356
Epoch 59: val_loss did not improve from 0.80673
22116/22116             84s 4ms/step
- accuracy: 0.7245 - loss: 0.8356 - val_accuracy: 0.7382 - val_loss: 0.8070
Epoch 60/100
22102/22116             0s 3ms/step -
accuracy: 0.7247 - loss: 0.8346
Epoch 60: val_loss did not improve from 0.80673
22116/22116             83s 4ms/step
- accuracy: 0.7247 - loss: 0.8346 - val_accuracy: 0.7360 - val_loss: 0.8074
Epoch 61/100
22107/22116             0s 3ms/step -
accuracy: 0.7241 - loss: 0.8352
Epoch 61: val_loss improved from 0.80673 to 0.80237, saving model to
/kaggle/working/best_cnn_model.keras
22116/22116             85s 4ms/step
- accuracy: 0.7241 - loss: 0.8352 - val_accuracy: 0.7369 - val_loss: 0.8024
Epoch 62/100
22102/22116             0s 3ms/step -
accuracy: 0.7243 - loss: 0.8344
Epoch 62: val_loss did not improve from 0.80237
22116/22116             87s 4ms/step
- accuracy: 0.7243 - loss: 0.8344 - val_accuracy: 0.7350 - val_loss: 0.8045
Epoch 63/100
22113/22116             0s 3ms/step -
accuracy: 0.7258 - loss: 0.8344
Epoch 63: val_loss did not improve from 0.80237
22116/22116             83s 4ms/step
- accuracy: 0.7258 - loss: 0.8344 - val_accuracy: 0.7377 - val_loss: 0.8089
Epoch 64/100
22115/22116             0s 3ms/step -
accuracy: 0.7253 - loss: 0.8333
Epoch 64: val_loss did not improve from 0.80237
```

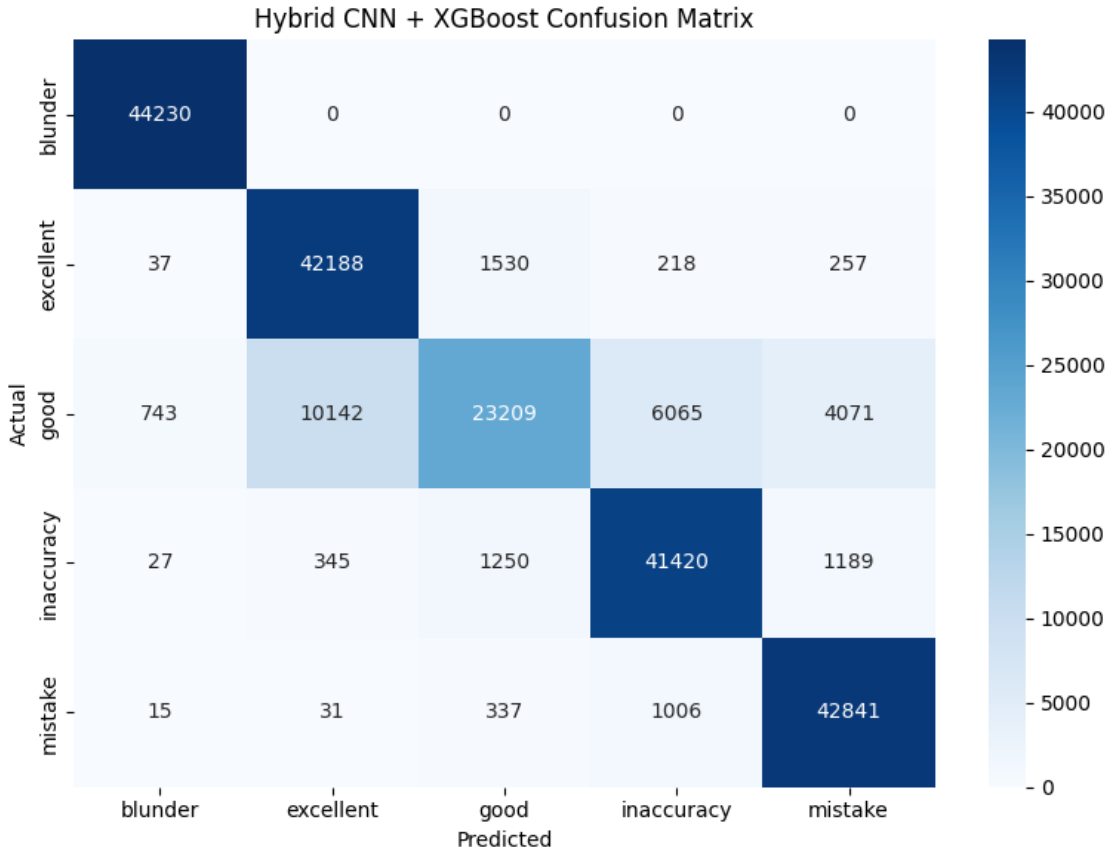
```
22116/22116            83s 4ms/step
- accuracy: 0.7253 - loss: 0.8333 - val_accuracy: 0.7361 - val_loss: 0.8057
Epoch 65/100
22104/22116            0s 3ms/step -
accuracy: 0.7252 - loss: 0.8341
Epoch 65: val_loss did not improve from 0.80237
22116/22116            83s 4ms/step
- accuracy: 0.7252 - loss: 0.8342 - val_accuracy: 0.7355 - val_loss: 0.8038
Epoch 66/100
22102/22116            0s 3ms/step -
accuracy: 0.7252 - loss: 0.8338
Epoch 66: val_loss did not improve from 0.80237
22116/22116            84s 4ms/step
- accuracy: 0.7252 - loss: 0.8338 - val_accuracy: 0.7357 - val_loss: 0.8063
Epoch 67/100
22112/22116            0s 3ms/step -
accuracy: 0.7252 - loss: 0.8334
Epoch 67: val_loss did not improve from 0.80237
22116/22116            84s 4ms/step
- accuracy: 0.7252 - loss: 0.8334 - val_accuracy: 0.7354 - val_loss: 0.8089
Epoch 68/100
22101/22116            0s 3ms/step -
accuracy: 0.7257 - loss: 0.8327
Epoch 68: val_loss did not improve from 0.80237
22116/22116            83s 4ms/step
- accuracy: 0.7257 - loss: 0.8327 - val_accuracy: 0.7348 - val_loss: 0.8035
Epoch 69/100
22111/22116            0s 3ms/step -
accuracy: 0.7258 - loss: 0.8318
Epoch 69: val_loss did not improve from 0.80237
22116/22116            83s 4ms/step
- accuracy: 0.7258 - loss: 0.8318 - val_accuracy: 0.7366 - val_loss: 0.8060
Epoch 70/100
22112/22116            0s 3ms/step -
accuracy: 0.7249 - loss: 0.8317
Epoch 70: val_loss did not improve from 0.80237
22116/22116            84s 4ms/step
- accuracy: 0.7249 - loss: 0.8317 - val_accuracy: 0.7349 - val_loss: 0.8086
Epoch 71/100
22108/22116            0s 3ms/step -
accuracy: 0.7260 - loss: 0.8311
Epoch 71: val_loss did not improve from 0.80237
22116/22116            83s 4ms/step
- accuracy: 0.7260 - loss: 0.8311 - val_accuracy: 0.7351 - val_loss: 0.8028

CNN model saved.
```

```
Extracting TRAIN deep features…

Processed 1024/884604
Processed 21504/884604
Processed 41984/884604
Processed 62464/884604
Processed 82944/884604
Processed 103424/884604
Processed 123904/884604
Processed 144384/884604
Processed 164864/884604
Processed 185344/884604
Processed 205824/884604
Processed 226304/884604
Processed 246784/884604
Processed 267264/884604
Processed 287744/884604
Processed 308224/884604
Processed 328704/884604
Processed 349184/884604
Processed 369664/884604
Processed 390144/884604
Processed 410624/884604
Processed 431104/884604
Processed 451584/884604
Processed 472064/884604
Processed 492544/884604
Processed 513024/884604
Processed 533504/884604
Processed 553984/884604
Processed 574464/884604
Processed 594944/884604
Processed 615424/884604
Processed 635904/884604
Processed 656384/884604
Processed 676864/884604
Processed 697344/884604
Processed 717824/884604
Processed 738304/884604
Processed 758784/884604
Processed 779264/884604
Processed 799744/884604
Processed 820224/884604
Processed 840704/884604
Processed 861184/884604
Processed 881664/884604

Extracting TEST deep features…
```

```
Processed 1024/221151
Processed 21504/221151
Processed 41984/221151
Processed 62464/221151
Processed 82944/221151
Processed 103424/221151
Processed 123904/221151
Processed 144384/221151
Processed 164864/221151
Processed 185344/221151
Processed 205824/221151

CNN features saved.


Starting XGBoost hyperparameter tuning…

Fitting 3 folds for each of 10 candidates, totalling 30 fits

/usr/local/lib/python3.12/dist-packages/xgboost/core.py:751: UserWarning:
[20:30:04] WARNING: /__w/xgboost/xgboost/src/common/error_msg.cc:62: Falling
back to prediction using DMatrix due to mismatched devices. This might lead to
higher memory usage and slower performance. XGBoost is running on: cuda:0, while
the input data is on: cpu.
Potential solutions:
- Use a data structure that matches the device ordinal in the booster.
- Set the device for booster before call to inplace_predict.

This warning will only be shown once.

  return func(**kwargs)

[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=8,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.4min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=8,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.4min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=8,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.4min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  55.1s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  54.5s
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  54.9s
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=1.0; total time=  54.0s
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=1.0; total time=  54.1s
```

```
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=1.0; total time=  53.8s
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 2.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 2.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 2.5min
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.8; total time=  54.1s
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.8; total time=  54.1s
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.05, max_depth=6,
n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.8; total time=  53.8s
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=1, subsample=0.8; total time= 1.5min
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 1.4min
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 1.4min
[CV] END colsample_bytree=0.8, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=300, reg_alpha=0, reg_lambda=1, subsample=1.0; total time= 1.4min
[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.03, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  52.8s
[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.03, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  52.6s
[CV] END colsample_bytree=0.8, gamma=0.1, learning_rate=0.03, max_depth=8,
n_estimators=300, reg_alpha=0, reg_lambda=2, subsample=0.8; total time=  52.8s
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=2, subsample=1.0; total time= 1.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=2, subsample=1.0; total time= 1.5min
[CV] END colsample_bytree=1.0, gamma=0.1, learning_rate=0.05, max_depth=10,
n_estimators=300, reg_alpha=0.5, reg_lambda=2, subsample=1.0; total time= 1.5min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=2, subsample=0.8; total time= 2.5min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=2, subsample=0.8; total time= 2.5min
[CV] END colsample_bytree=1.0, gamma=0, learning_rate=0.03, max_depth=10,
n_estimators=500, reg_alpha=0.5, reg_lambda=2, subsample=0.8; total time= 2.5min

Best Parameters:

{'subsample': 0.8, 'reg_lambda': 1, 'reg_alpha': 0.5, 'n_estimators': 500,
```

```
'max_depth': 10, 'learning_rate': 0.03, 'gamma': 0.1, 'colsample_bytree': 1.0}

XGBoost model saved.

Evaluating model…

Hybrid Model Accuracy: 87.67%
```



Classification Report:

	precision	recall	f1-score	support
blunder	0.98	1.00	0.99	44230
excellent	0.80	0.95	0.87	44230
good	0.88	0.52	0.66	44230
inaccuracy	0.85	0.94	0.89	44231
mistake	0.89	0.97	0.93	44230

```
    accuracy                           0.88    221151
   macro avg       0.88      0.88      0.87    221151
weighted avg       0.88      0.88      0.87    221151
```

```
Everything saved successfully.
```

```
Saved files location:

/kaggle/working/
```

1.1 model summary

[10]:
```python
# ============================================================
# IMPORTS
# ============================================================

import numpy as np
import tensorflow as tf

from tensorflow.keras import layers
from tensorflow.keras import models
from tensorflow.keras import regularizers

from tensorflow.keras.utils import plot_model

from IPython.display import Image
from IPython.display import display

# ============================================================
# ASSUMED INPUT DETAILS
# ============================================================

# tensor shape from chess tensor
tensor_shape = (8, 8, 12)

# scalar feature count
scalar_dim = 15

# number of classes
num_classes = 5

# ============================================================
# BUILD CNN FEATURE EXTRACTOR MODEL
# ============================================================
```

```
def build_cnn_model(
    tensor_shape,
    scalar_dim,
    num_classes
):

    # ========================================================
    # TENSOR INPUT BRANCH
    # ========================================================

    tensor_input = layers.Input(
        shape=tensor_shape,
        name="Tensor_Input"
    )

    x = layers.Conv2D(
        32,
        (3,3),
        padding='same',
        activation='swish',
        name="Conv2D_1"
    )(tensor_input)

    x = layers.BatchNormalization(
        name="BatchNorm_1"
    )(x)

    x = layers.MaxPooling2D(
        (2,2),
        name="MaxPool_1"
    )(x)

    x = layers.Conv2D(
        64,
        (3,3),
        padding='same',
        activation='swish',
        name="Conv2D_2"
    )(x)

    x = layers.BatchNormalization(
        name="BatchNorm_2"
    )(x)

    x = layers.Flatten(
        name="Flatten"
```

```
    )(x)

    cnn_features = layers.Dense(
        128,
        activation='swish',
        kernel_regularizer=regularizers.l2(0.001),
        name="Deep_Features"
    )(x)

    x = layers.Dropout(
        0.3,
        name="CNN_Dropout"
    )(cnn_features)

    # ========================================================
    # SCALAR INPUT BRANCH
    # ========================================================

    scalar_input = layers.Input(
        shape=(scalar_dim,),
        name="Scalar_Input"
    )

    y = layers.Dense(
        64,
        activation='swish',
        name="Scalar_Dense"
    )(scalar_input)

    y = layers.BatchNormalization(
        name="Scalar_BatchNorm"
    )(y)

    y = layers.Dropout(
        0.2,
        name="Scalar_Dropout"
    )(y)

    # ========================================================
    # MERGE BRANCHES
    # ========================================================

    combined = layers.concatenate(
        [x, y],
        name="Concatenate"
    )
```

```
    z = layers.Dense(
        128,
        activation='swish',
        name="Combined_Dense"
    )(combined)

    z = layers.Dropout(
        0.3,
        name="Combined_Dropout"
    )(z)

    output = layers.Dense(
        num_classes,
        activation='softmax',
        name="Output_Layer"
    )(z)

    # =========================================================
    # CREATE MODEL
    # =========================================================

    model = models.Model(
        inputs=[tensor_input, scalar_input],
        outputs=output,
        name="Chess_Hybrid_CNN_Model"
    )

    # =========================================================
    # COMPILE MODEL
    # =========================================================

    optimizer = tf.keras.optimizers.Adam(
        learning_rate=0.0005,
        clipnorm=1.0
    )

    model.compile(
        optimizer=optimizer,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model

# ============================================================
# BUILD MODEL
# ============================================================
```

```python
model = build_cnn_model(
    tensor_shape=tensor_shape,
    scalar_dim=scalar_dim,
    num_classes=num_classes
)

# ============================================================
# COMPLETE MODEL SUMMARY
# ============================================================

print("\n========== COMPLETE MODEL SUMMARY ==========\n")

model.summary(
    line_length=140,
    expand_nested=True,
    show_trainable=True
)

# ============================================================
# MODEL INPUTS
# ============================================================

print("\n========== MODEL INPUTS ==========\n")

for i, inp in enumerate(model.inputs):

    print(f"Input {i+1}")

    print(f"Name  : {inp.name}")

    print(f"Shape : {inp.shape}")

    print()

# ============================================================
# MODEL OUTPUT
# ============================================================

print("\n========== MODEL OUTPUT ==========\n")

print(f"Output Shape : {model.output.shape}")

# ============================================================
# LAYER DETAILS
# ============================================================
```

```
print("\n========== LAYER-WISE DETAILS ==========\n")

for i, layer in enumerate(model.layers):

    print(f"\nLayer {i+1}")

    print(f"Name         : {layer.name}")

    print(f"Type         : {layer.__class__.__name__}")

    try:
        print(f"Input Shape  : {layer.input.shape}")
    except:
        pass

    try:
        print(f"Output Shape : {layer.output.shape}")
    except:
        pass

    print(f"Parameters   : {layer.count_params()}")

    print(f"Trainable    : {layer.trainable}")

# ============================================================
# PARAMETER SUMMARY
# ============================================================

print("\n========== PARAMETER SUMMARY ==========\n")

print(f"Total Parameters      : {model.count_params():,}")

trainable_params = np.sum(
    [np.prod(v.shape) for v in model.trainable_weights]
)

non_trainable_params = np.sum(
    [np.prod(v.shape) for v in model.non_trainable_weights]
)

print(f"Trainable Parameters  : {trainable_params:,}")

print(f"Non-Trainable Params  : {non_trainable_params:,}")

# ============================================================
# SAVE MODEL ARCHITECTURE IMAGE
# ============================================================
```

```
plot_model(
    model,
    to_file="hybrid_cnn_model_architecture.png",
    show_shapes=True,
    show_dtype=True,
    show_layer_names=True,
    expand_nested=True,
    dpi=70
)

print("\nModel architecture image saved as:")
print("hybrid_cnn_model_architecture.png")

# ============================================================
# DISPLAY MODEL IMAGE
# ============================================================

display(
    Image(
        filename="hybrid_cnn_model_architecture.png"
    )
)
```

```
========== COMPLETE MODEL SUMMARY ==========

Model: "Chess_Hybrid_CNN_Model"


 Layer (type)                        Output Shape                      ␣
↪  Param #   Connected to

 Tensor_Input (InputLayer)           (None, 8, 8, 12)                  ␣
↪        0   -

 Conv2D_1 (Conv2D)                   (None, 8, 8, 32)                  ␣
↪    3,488   Tensor_Input[0][0]

 BatchNorm_1 (BatchNormalization)    (None, 8, 8, 32)                  ␣
↪      128   Conv2D_1[0][0]

 MaxPool_1 (MaxPooling2D)            (None, 4, 4, 32)                  ␣
↪        0   BatchNorm_1[0][0]
```

```
Conv2D_2 (Conv2D)                (None, 4, 4, 64)                ␣
↪    18,496   MaxPool_1[0][0]

BatchNorm_2 (BatchNormalization)  (None, 4, 4, 64)                ␣
↪       256   Conv2D_2[0][0]

Scalar_Input (InputLayer)        (None, 15)                      ␣
↪         0   -

Flatten (Flatten)                (None, 1024)                    ␣
↪         0   BatchNorm_2[0][0]

Scalar_Dense (Dense)             (None, 64)                      ␣
↪     1,024   Scalar_Input[0][0]

Deep_Features (Dense)            (None, 128)                     ␣
↪   131,200   Flatten[0][0]

Scalar_BatchNorm                 (None, 64)                      ␣
↪       256   Scalar_Dense[0][0]
(BatchNormalization)                                             ␣
↪

CNN_Dropout (Dropout)            (None, 128)                     ␣
↪         0   Deep_Features[0][0]

Scalar_Dropout (Dropout)         (None, 64)                      ␣
↪         0   Scalar_BatchNorm[0][

Concatenate (Concatenate)        (None, 192)                     ␣
↪         0   CNN_Dropout[0][0],
                                                                 ␣
↪             Scalar_Dropout[0][0]

Combined_Dense (Dense)           (None, 128)                     ␣
↪    24,704   Concatenate[0][0]

Combined_Dropout (Dropout)       (None, 128)                     ␣
↪         0   Combined_Dense[0][0]

Output_Layer (Dense)             (None, 5)                       ␣
↪       645   Combined_Dropout[0][


Total params: 180,197 (703.89 KB)
```

Trainable params: 179,877 (702.64 KB)

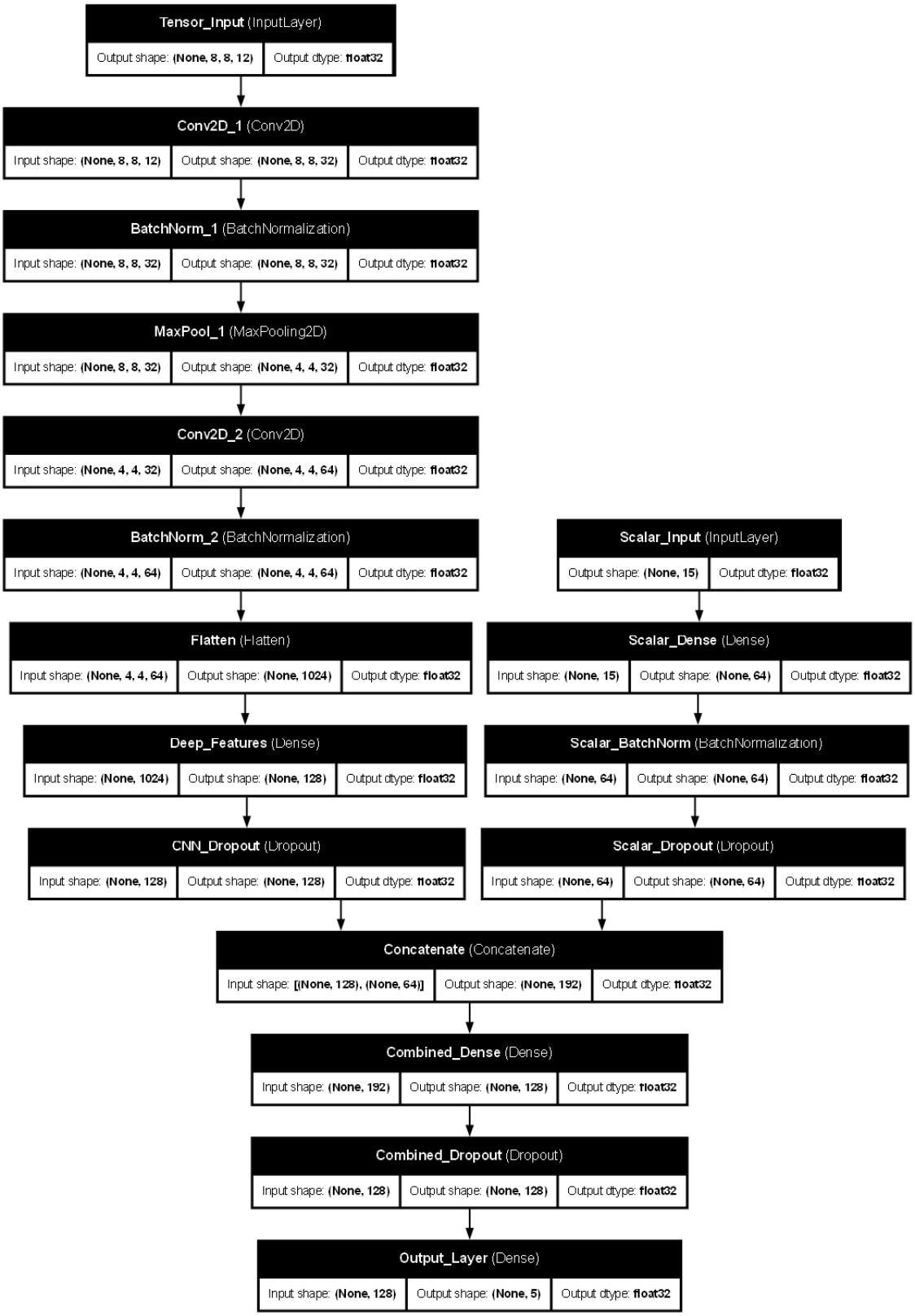
Non-trainable params: 320 (1.25 KB)

```
========== MODEL INPUTS ==========

Input 1
Name  : Tensor_Input
Shape : (None, 8, 8, 12)

Input 2
Name  : Scalar_Input
Shape : (None, 15)


========== MODEL OUTPUT ==========

Output Shape : (None, 5)

========== LAYER-WISE DETAILS ==========


Layer 1
Name         : Tensor_Input
Type         : InputLayer
Output Shape : (None, 8, 8, 12)
Parameters   : 0
Trainable    : True

Layer 2
Name         : Conv2D_1
Type         : Conv2D
Input Shape  : (None, 8, 8, 12)
Output Shape : (None, 8, 8, 32)
Parameters   : 3488
Trainable    : True

Layer 3
Name         : BatchNorm_1
Type         : BatchNormalization
Input Shape  : (None, 8, 8, 32)
Output Shape : (None, 8, 8, 32)
Parameters   : 128
Trainable    : True

Layer 4
```

```
Name         : MaxPool_1
Type         : MaxPooling2D
Input Shape  : (None, 8, 8, 32)
Output Shape : (None, 4, 4, 32)
Parameters   : 0
Trainable    : True

Layer 5
Name         : Conv2D_2
Type         : Conv2D
Input Shape  : (None, 4, 4, 32)
Output Shape : (None, 4, 4, 64)
Parameters   : 18496
Trainable    : True

Layer 6
Name         : BatchNorm_2
Type         : BatchNormalization
Input Shape  : (None, 4, 4, 64)
Output Shape : (None, 4, 4, 64)
Parameters   : 256
Trainable    : True

Layer 7
Name         : Scalar_Input
Type         : InputLayer
Output Shape : (None, 15)
Parameters   : 0
Trainable    : True

Layer 8
Name         : Flatten
Type         : Flatten
Input Shape  : (None, 4, 4, 64)
Output Shape : (None, 1024)
Parameters   : 0
Trainable    : True

Layer 9
Name         : Scalar_Dense
Type         : Dense
Input Shape  : (None, 15)
Output Shape : (None, 64)
Parameters   : 1024
Trainable    : True

Layer 10
Name         : Deep_Features
```

```
Type          : Dense
Input Shape   : (None, 1024)
Output Shape  : (None, 128)
Parameters    : 131200
Trainable     : True

Layer 11
Name          : Scalar_BatchNorm
Type          : BatchNormalization
Input Shape   : (None, 64)
Output Shape  : (None, 64)
Parameters    : 256
Trainable     : True

Layer 12
Name          : CNN_Dropout
Type          : Dropout
Input Shape   : (None, 128)
Output Shape  : (None, 128)
Parameters    : 0
Trainable     : True

Layer 13
Name          : Scalar_Dropout
Type          : Dropout
Input Shape   : (None, 64)
Output Shape  : (None, 64)
Parameters    : 0
Trainable     : True

Layer 14
Name          : Concatenate
Type          : Concatenate
Output Shape  : (None, 192)
Parameters    : 0
Trainable     : True

Layer 15
Name          : Combined_Dense
Type          : Dense
Input Shape   : (None, 192)
Output Shape  : (None, 128)
Parameters    : 24704
Trainable     : True

Layer 16
Name          : Combined_Dropout
Type          : Dropout
```

```
Input Shape  : (None, 128)
Output Shape : (None, 128)
Parameters   : 0
Trainable    : True

Layer 17
Name         : Output_Layer
Type         : Dense
Input Shape  : (None, 128)
Output Shape : (None, 5)
Parameters   : 645
Trainable    : True

========== PARAMETER SUMMARY ==========

Total Parameters      : 180,197
Trainable Parameters  : 179,877
Non-Trainable Params  : 320

Model architecture image saved as:
hybrid_cnn_model_architecture.png
```


Tensor_Input (InputLayer)
Output shape: (None, 8, 8, 12)
Output dtype: float32
Conv2D_1 (Conv2D)
Input shape: (None, 8, 8, 12)
Output shape: (None, 8, 8, 32)
Output dtype: float32
BatchNorm_1 (BatchNormalization)
Input shape: (None, 8, 8, 32)
Output shape: (None, 8, 8, 32)
Output dtype: float32
MaxPool_1 (MaxPooling2D)
Input shape: (None, 8, 8, 32)
Output shape: (None, 4, 4, 32)
Output dtype: float32
Conv2D_2 (Conv2D)
Input shape: (None, 4, 4, 32)
Output shape: (None, 4, 4, 64)
Output dtype: float32
BatchNorm_2 (BatchNormalization)
Input shape: (None, 4, 4, 64)
Output shape: (None, 4, 4, 64)
Output dtype: float32
Flatten (Flatten)
Input shape: (None, 4, 4, 64)
Output shape: (None, 1024)
Output dtype: float32
Deep_Features (Dense)
Input shape: (None, 1024)
Output shape: (None, 128)
Output dtype: float32
CNN_Dropout (Dropout)
Input shape: (None, 128)
Output shape: (None, 128)
Output dtype: float32
Scalar_Input (InputLayer)
Output shape: (None, 15)
Output dtype: float32
Scalar_Dense (Dense)
Input shape: (None, 15)
Output shape: (None, 64)
Output dtype: float32
Scalar_BatchNorm (BatchNormalization)
Input shape: (None, 64)
Output shape: (None, 64)
Output dtype: float32
Scalar_Dropout (Dropout)
Input shape: (None, 64)
Output shape: (None, 64)
Output dtype: float32
Concatenate (Concatenate)
Input shape: [(None, 128), (None, 64)]
Output shape: (None, 192)
Output dtype: float32
Combined_Dense (Dense)
Input shape: (None, 192)
Output shape: (None, 128)
Output dtype: float32
Combined_Dropout (Dropout)
Input shape: (None, 128)
Output shape: (None, 128)
Output dtype: float32
Output_Layer (Dense)
Input shape: (None, 128)
Output shape: (None, 5)
Output dtype: float32

[9]:
```
print("\n========== COMPLETE HYBRID PIPELINE ==========\n")

print("""
12×8×8 Chess Tensor



        ↓               ↓
CNN Branch         Scalar Feature Branch
(Conv2D Layers)    (Dense Layers)

        ↓               ↓
Tensor Feature Maps    Scalar Embeddings


                ↓
Feature Concatenation
                ↓
Dense Feature Fusion Layer
                ↓
128 Deep Hybrid Features
                ↓
Extracted Deep Features
                +
Original Scalar Features
                ↓
143 Hybrid Features
                ↓
XGBoost Classifier
                ↓
Final Class Prediction
""")
```

```
========== COMPLETE HYBRID PIPELINE ==========


12×8×8 Chess Tensor



        ↓               ↓
CNN Branch         Scalar Feature Branch
(Conv2D Layers)    (Dense Layers)

        ↓               ↓
Tensor Feature Maps    Scalar Embeddings
```

```
                ↓
Feature Concatenation
                ↓
Dense Feature Fusion Layer
                ↓
128 Deep Hybrid Features
                ↓
Extracted Deep Features
                +
Original Scalar Features
                ↓
143 Hybrid Features
                ↓
XGBoost Classifier
                ↓
Final Class Prediction
```

[ ]: